

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 6

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Command History Storage and Capacity Management

Question:

Store commands in a history buffer, maintain command history, handle capacity limits, display stored history, and validate history consistency.

Source Code:

```
haasini@SSH:~/OSSP_upload/Skill_Week6$ cat > T1.c <<'EOF'
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_LEN 100

int main(void)
{
    char history[MAX_HISTORY][MAX_LEN];
    char command[MAX_LEN];

    int count = 0;
    int total = 0;

    printf("=== Command History Demo ===\n");
    printf("Maximum history capacity: %d commands\n", MAX_HISTORY);
    printf("Type 'history' to display history\n");
    printf("Type 'exit' to stop\n\n");

    while (1)
    {
        printf("shell> ");

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
            EOF return 0;
        history program terminated.\n")
haasini@SSH:~/OSSP_upload/Skill_Week6$ |
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week6$ ./T1
=== Command History Demo ===
Maximum history capacity: 5 commands
Type 'history' to display history
Type 'exit' to stop

shell> ls
shell> pwd
shell> date
shell> whoami
shell> uname
shell> echo
shell> history

Stored Command History:
2. pwd
3. date
4. whoami
5. uname
6. echo

shell> exit
History program terminated.
```

Observations:

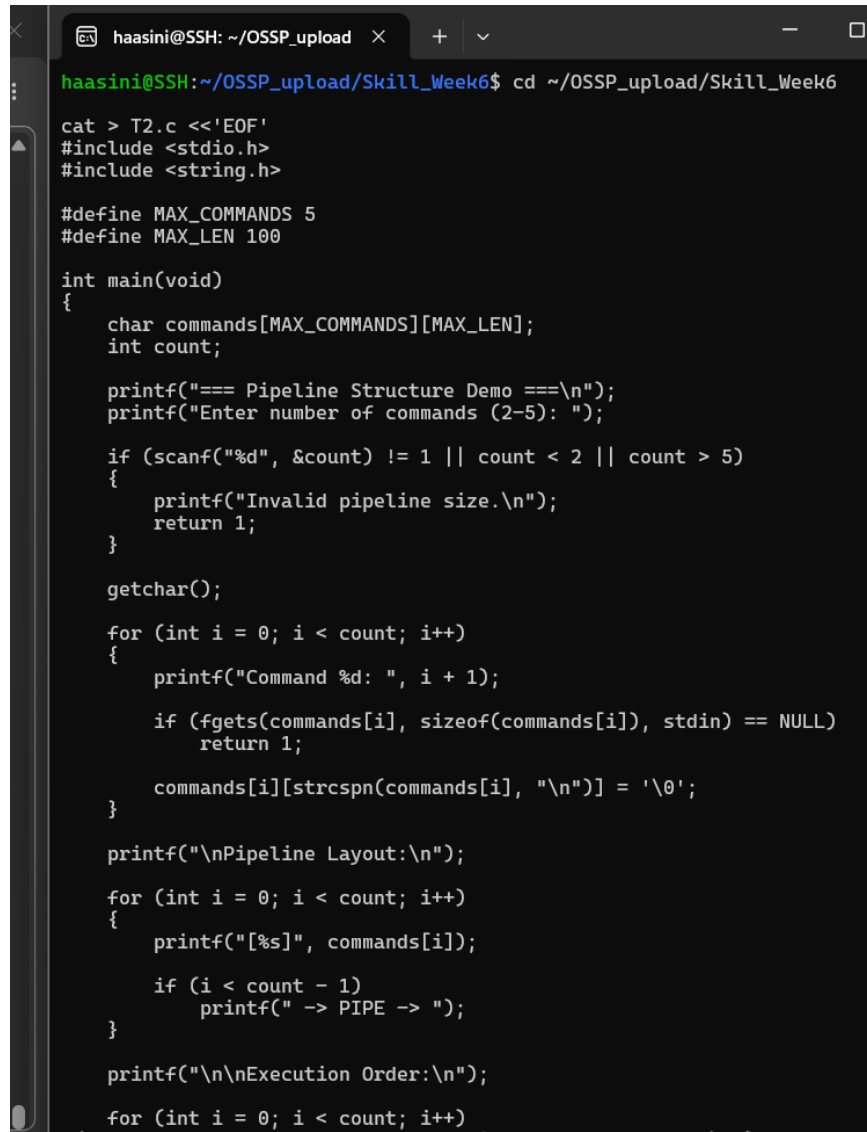
1. Commands are stored in a fixed-size 2D array `history[MAX_HISTORY][MAX_LEN]` with a maximum capacity of 5 commands.
2. `MAX_HISTORY` (5) defines the circular buffer size, and `MAX_LEN` (100) limits individual command length.
3. The count variable tracks the number of commands currently stored, incrementing with each new command entry.
4. The total variable tracks cumulative commands entered, enabling detection of when capacity is exceeded and circular overwriting begins.
5. When total exceeds `MAX_HISTORY`, the oldest command is overwritten using modulo indexing (`total % MAX_HISTORY`) to implement circular buffer behavior.
6. The 'history' command displays all retained commands in sequential order, demonstrating the current buffer state.
7. The program demonstrates capacity handling by storing six commands (`ls`, `pwd`, `date`, `whoami`, `uname`, `echo`), with the oldest (`ls`) being overwritten.
8. The 'exit' command terminates the loop cleanly, displaying the termination message 'History program terminated.'

Task 2: Pipeline Structure and Execution Order

Question:

Create a pipeline structure to store multiple commands, maintain their execution order, represent connections between command stages, and validate the pipeline layout.

Source Code:

A screenshot of a terminal window with a dark background. The window title is 'haasini@SSH: ~/OSSP_upload'. The prompt is 'haasini@SSH:~/OSSP_upload/Skill_Week6\$'. The user has entered 'cd ~/OSSP_upload/Skill_Week6'. The code being displayed is a C program named 'T2.c'. It defines 'MAX_COMMANDS' as 5 and 'MAX_LEN' as 100. The 'main' function prompts the user to enter the number of commands (2-5). It validates the input, reads the commands into an array, and then prints the pipeline layout and execution order. The code is as follows:

```
cat > T2.c <<'EOF'
#include <stdio.h>
#include <string.h>

#define MAX_COMMANDS 5
#define MAX_LEN 100

int main(void)
{
    char commands[MAX_COMMANDS][MAX_LEN];
    int count;

    printf("=== Pipeline Structure Demo ===\n");
    printf("Enter number of commands (2-5): ");

    if (scanf("%d", &count) != 1 || count < 2 || count > 5)
    {
        printf("Invalid pipeline size.\n");
        return 1;
    }

    getchar();

    for (int i = 0; i < count; i++)
    {
        printf("Command %d: ", i + 1);

        if (fgets(commands[i], sizeof(commands[i]), stdin) == NULL)
            return 1;

        commands[i][strcspn(commands[i], "\n")] = '\0';
    }

    printf("\nPipeline Layout:\n");

    for (int i = 0; i < count; i++)
    {
        printf("[%s]", commands[i]);

        if (i < count - 1)
            printf(" -> PIPE -> ");
    }

    printf("\n\nExecution Order:\n");

    for (int i = 0; i < count; i++)
```

Output:

```

    for (int i = 0; i < count; i++)
    ./T2-Wall -Wextra T2.c -o T2ture validated successfully.\n");
=== Pipeline Structure Demo ===
Enter number of commands (2-5): 3
Command 1: ls -l
Command 2: grep .c
Command 3: wc -l

Pipeline Layout:
[ls -l] -> PIPE -> [grep .c] -> PIPE -> [wc -l]

Execution Order:
Stage 1: ls -l
Stage 2: grep .c
Stage 3: wc -l

Pipeline structure validated successfully.
haasini@SSH:~/OSSP_upload/Skill_Week6$ |

```

Observations:

1. The program stores multiple commands in a two-dimensional command array `commands[MAX_COMMANDS][MAX_LEN]` with a maximum capacity of 5 commands.
2. The user specifies the number of pipeline commands to be created, with input prompted as 'Enter number of commands (2-5):'.
3. The program validates that the pipeline contains between 2 and 5 commands using range checking (`count < 2 || count > 5`), rejecting invalid pipeline sizes with 'Invalid pipeline size.'
4. Each command is stored as a separate pipeline stage, read from user input using `fgets()` and stored with newline removed using `strcspn()`.
5. The commands maintain the same execution order in which they were entered, displayed sequentially from stage 1 to stage n.
6. The program displays the pipeline layout by printing each command followed by ' -> PIPE -> ' to represent logical connections between adjacent command stages.
7. For the demonstrated input of 3 commands, the pipeline layout displays as: `[ls -l] -> PIPE -> [grep .c] -> PIPE -> [wc -l]`.
8. The execution order section explicitly labels each command with its stage number: Stage 1: `ls -l`, Stage 2: `grep .c`, Stage 3: `wc -l`.
9. The successful validation message 'Pipeline structure validated successfully.' confirms that the pipeline structure was created and validated correctly.

Task 3: Two-Stage Command Pipeline Using pipe() and dup2()

Question:

Create a two-stage command pipeline using pipe(), fork(), dup2(), and exec(). Redirect the output of one process to the input of another process, close unused file descriptors, and synchronize child processes.

Source Code:

```
history retrieval successful.
haasini@SSH:~/OSSP_upload/Skill_Week6$ cat > T3.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int pipefd[2];

    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        return 1;
    }

    pid_t first = fork();

    if (first < 0)
    {
        perror("fork");
        return 1;
    }

    if (first == 0)
    {
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid_t second = fork();

    if (second < 0)
    {
        perror("fork");
        return 1;
    }

    EOF return 0;\nTwo-stage pipeline completed successfully.\n");
```

Output:

```

    for return 0, two stage pipeline completed successfully. (n);
haasini@SSH:~/OSSP_upload/Skill_Week6$ gcc -Wall -Wextra T3.c -o T3
haasini@SSH:~/OSSP_upload/Skill_Week6$ ls -l | grep ".c"
-rw-r--r-- 1 haasini haasini 1296 Sep 12 09:23 T1.c
-rw-r--r-- 1 haasini haasini 884 Sep 12 09:26 T2.c
-rw-r--r-- 1 haasini haasini 1024 Sep 12 09:28 T3.c
haasini@SSH:~/OSSP_upload/Skill_Week6$ ./T3
-rw-r--r-- 1 haasini haasini 1296 Sep 12 09:23 T1.c
-rw-r--r-- 1 haasini haasini 884 Sep 12 09:26 T2.c
-rw-r--r-- 1 haasini haasini 1024 Sep 12 09:28 T3.c

Two-stage pipeline completed successfully.
haasini@SSH:~/OSSP_upload/Skill_Week6$ |

```

Observations:

1. pipe() creates a unidirectional communication channel with pipefd[0] as the read end and pipefd[1] as the write end.
2. Two fork() calls create separate child processes: the first executes 'ls -l' and the second executes 'grep ".c"'.
".c".
3. The first child process redirects its standard output (file descriptor 1) to the pipe write end using dup2(pipefd[1], STDOUT_FILENO).
4. The second child process redirects its standard input (file descriptor 0) from the pipe read end using dup2(pipefd[0], STDIN_FILENO).
5. Each child closes both pipe file descriptors after redirection to prevent resource leaks and enable proper EOF signaling.
6. execlp() replaces each child process with the target command: the first with 'ls -l' and the second with 'grep ".c"'.
'grep ".c"'.
".c".
7. The parent process closes its copies of both pipe file descriptors, leaving only the children with active pipe access.
8. The output demonstrates successful pipeline execution showing three .c files (T1.c, T2.c, T3.c) filtered from the full 'ls -l' listing.
9. The completion message 'Two-stage pipeline completed successfully.' confirms proper synchronization of both child processes.

Task 4: Multi-Stage Pipeline Using Multiple Pipes

Question:

Create a multi-stage command pipeline using multiple pipes and multiple child processes. Coordinate execution, connect standard input/output between stages, close unused descriptors, and wait for all child processes.

Source Code:



```
haasini@SSH: ~/OSSP_upload × + v
GNU nano 7.2 T4.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    int pipe1[2];
    int pipe2[2];

    if (pipe(pipe1) == -1)
    {
        perror("pipe1");
        return 1;
    }

    if (pipe(pipe2) == -1)
    {
        perror("pipe2");
        return 1;
    }

    pid_t p1 = fork();

    if (p1 < 0)
    {
        perror("fork");
        return 1;
    }

    if (p1 == 0)
    {
        dup2(pipe1[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("ls", "ls", "-l", NULL);

        perror("execlp ls");
        exit(1);
    }

    pid_t p2 = fork();

    if (p2 < 0)
    {
        perror("fork");
        return 1;
    }

    if (p2 == 0)
    {
        dup2(pipe1[0], STDIN_FILENO);
        dup2(pipe2[1], STDOUT_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);
    }
}
```

```

        close(pipe2[1]);

        execlp("grep", "grep", ".c", NULL);

        perror("execlp grep");
        exit(1);
    }

    pid_t p3 = fork();

    if (p3 < 0)
    {
        perror("fork");
        return 1;
    }

    if (p3 == 0)
    {
        dup2(pipe2[0], STDIN_FILENO);

        close(pipe1[0]);
        close(pipe1[1]);
        close(pipe2[0]);
        close(pipe2[1]);

        execlp("wc", "wc", "-l", NULL);

        perror("execlp wc");
        exit(1);
    }

    close(pipe1[0]);
    close(pipe1[1]);
    close(pipe2[0]);
    close(pipe2[1]);

    waitpid(p1, NULL, 0);
    waitpid(p2, NULL, 0);
    waitpid(p3, NULL, 0);

    printf("Three-stage pipeline completed successfully.\n");

    return 0;
}

```

Output:

```

haasini@SSH:~/OSSP_upload/Skill_Week6$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week6$ gcc -Wall -Wextra T4.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week6$ ls -l | grep ".c" | wc -l
4
haasini@SSH:~/OSSP_upload/Skill_Week6$ ./T4
4
Three-stage pipeline completed successfully.
haasini@SSH:~/OSSP_upload/Skill_Week6$ |

```

Observations:

1. Three child processes are required to execute the three commands: ls, grep, and wc in a multi-stage pipeline.
2. Two pipes (pipe1 and pipe2) are created to connect the three stages, with pipe1 connecting ls to grep and pipe2 connecting grep to wc.
3. The first child process (p1) redirects its standard output to pipe1's write end using dup2(pipe1[1], STDOUT_FILENO), then executes 'ls -l'.

4. The second child process (p2) redirects stdin from pipe1's read end and stdout to pipe2's write end, executing 'grep ".c"' to filter intermediate output.
5. The third child process (p3) redirects its standard input from pipe2's read end using dup2(pipe2[0], STDIN_FILENO), then executes 'wc -l'.
6. Each child process closes all pipe file descriptors after redirection to prevent resource leaks and enable proper EOF signaling across stages.
7. The parent process closes all pipe file descriptors before waiting, ensuring children have exclusive access to the pipeline.
8. Three separate waitpid() calls synchronize the parent with all three child processes, ensuring they complete before the program terminates.
9. The output displays '4', which is the count of .c files from the pipeline execution (ls -l | grep ".c" | wc -l).
10. The completion message 'Three-stage pipeline completed successfully.' confirms proper execution and synchronization of all three pipeline stages.